

ENGINEERING ENABLEMENT

Javacore Analyser

New Key Features

Recently added capabilities for faster performance investigations

Chris Kazmierczyk

3 September 2026

Verbose GC File Visualisation

HAR File Support

AI-Based Thread Classification

Tips Enhancements

Tool Integration via Plugin
Architecture

WatsonX AI Integration for Intelligent
Tips



1 / 12



What is WAIT2 Javacore Analyser?

PURPOSE

Analyses and visualises Java performance data

WAIT2 Javacore Analyser processes javacore files and related diagnostic data to help identify performance bottlenecks, thread hangs, memory issues and GC problems — without manual stack trace inspection.

WHAT IT ANALYSES

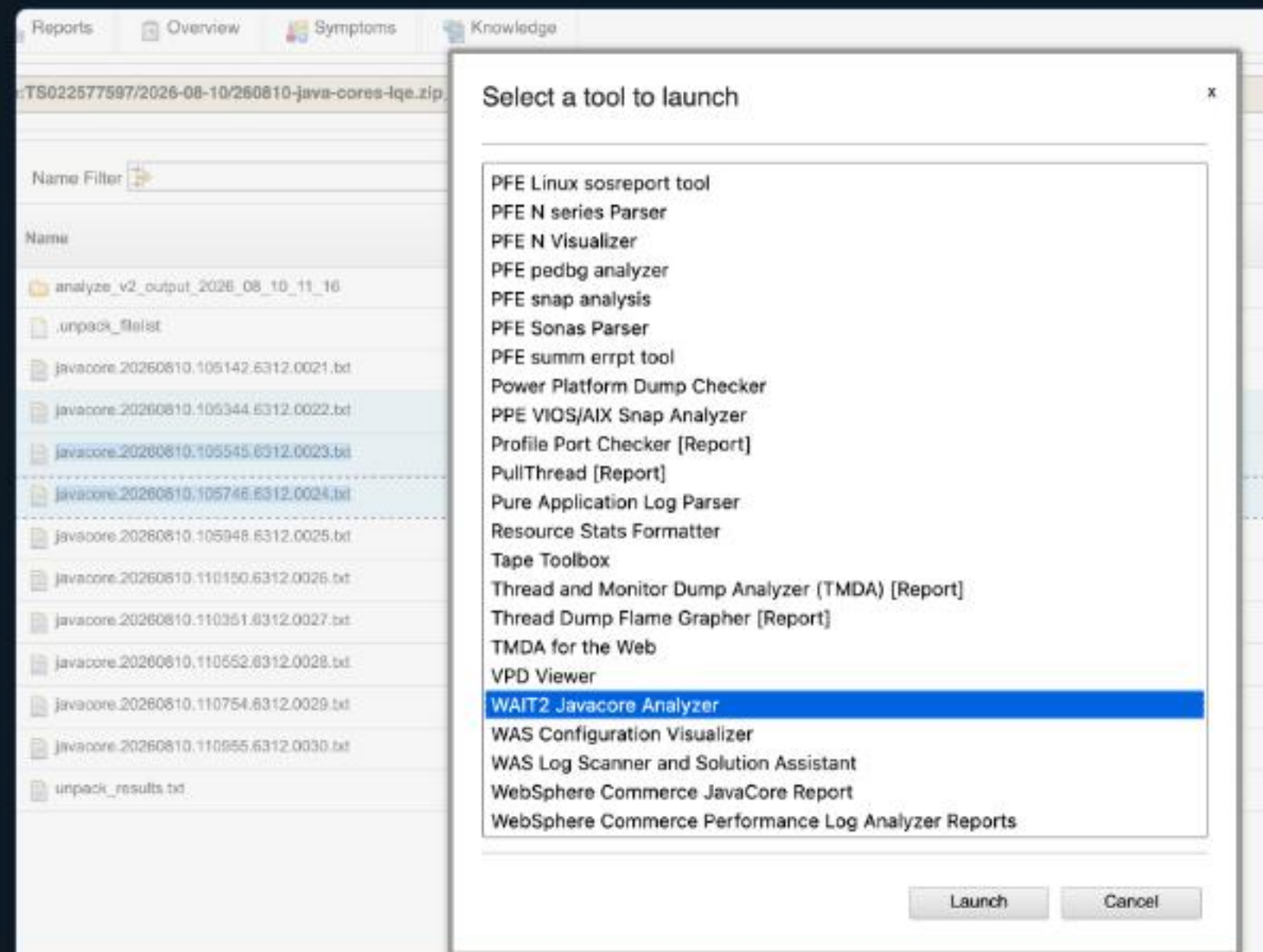
Multiple diagnostic data sources in one run

- ▶ **Javacore files** — thread states, CPU usage, lock contention
- ▶ **Verbose GC logs** — heap usage, pause times, collection frequency
- ▶ **HAR files** — HTTP call durations, status codes, slow endpoints

WHERE TO RUN IT

Available directly on Ecurep

Launch from the Ecurep tool list while browsing a customer case — no Citrix, no local setup, no file download required.



The Generated Report

WHAT

A self-contained diagnostic report

The tool produces a single HTML file with everything needed to diagnose the issue — no extra tools or viewers required.

HOW TO USE

Open in any browser

The report opens automatically after the run. Share it with colleagues.

OUTPUT HIGHLIGHTS

Rich, interactive content

- ▶ **Expandable sections** — navigate directly to the area of interest
- ▶ **Embedded documentation** — inline chart explainers
- ▶ **Intelligent tips** — identify potential causes of performance issues; warns when investigation results may not be reliable
- ▶ **Interactive charts** — zoom and hover to explore data

Javacore Analyser Report

from data between 2026-01-13 06:56:47 and 2026-01-13 07:20:33

[Input Files](#)

[System Information](#)

[Intelligent tips](#)

[System resources utilization](#)

Expandable sections

CPU Load

[What does this chart tell me?](#)

Embedded documentation

This chart shows the total CPU usage of all the threads in the javacore, expressed as percentage of all the processor cores. The maximum possible value is therefore 100%, which would indicate all the cores are completely busy.

Each bar represents one javacore in the data set. This value is computed incrementally with relation to the previous javacore, hence it is not available for the first javacore file.



Interactive charts

No verbosegc logs were provided, so verbose GC data cannot be shown.

Verbose GC File Visualisation

WHAT

Built-in verbose GC analysis

Verbose GC logs are parsed alongside javacores — heap usage, GC pause times and collection frequency are visualised without any external GC tool.

HOW TO USE

Include GC logs in your input

- ▶ Add verbose GC log files to your zip, folder or file list
- ▶ The tool auto-detects and processes them alongside javacores

OUTPUT

GC Activity chart in the report

- ▶ Heap usage, nursery and tenured memory over time
- ▶ GC pause durations overlaid as bars
- ▶ Long Stop-the-World pauses automatically flagged in Tips

System resources utilization

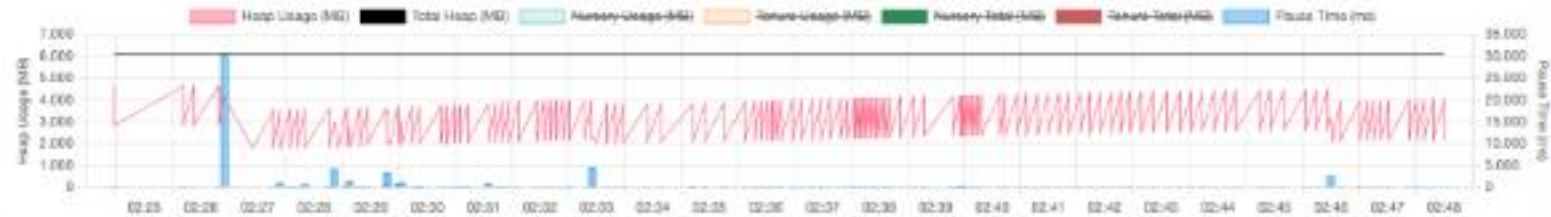
CPU Load

What does this chart tell me?



Garbage Collection Activity

What does this chart tell me?



CPU Load (top) and GC Activity (bottom) — heap usage, pause times and collection frequency shown together

Heap oscillates between ~2–4 GB (6 GB total). A single exceptionally long pause (~30 s) is visible at **02:26** — a Stop-the-World event that would cause a severe application stall.



HTTP traffic analysis inside the report

HAR (HTTP Archive) files capture browser/application network traffic. The tool parses them and surfaces slow calls, error codes and payload sizes — no external HAR viewer needed.

HOW TO USE

Include HAR files in your input

- ▶ Add *.har files to your zip, folder or file list
- ▶ Capture HAR from a browser DevTools session
- ▶ The tool auto-detects and includes them — no extra steps

OUTPUT

HTTP calls table in the report

- ▶ All HTTP calls listed in a table — sort column can be changed
- ▶ Expandable request / response details per call
- ▶ Plan to display long HAR requests or errored requests in Tips

[illegible]

All call durations are below **900 ms**, however one request fails with **HTTP 400** — this error response is a possible cause of the performance issue being investigated.

Automatic thread categorisation by a pre-trained model

Nothing extra — runs automatically

- ▶ Classification runs on every javacore analysis automatically
- ▶ No additional files or configuration required

Classification column + Over Time chart

- ▶ Classification column visible in the All Threads table
- ▶ Thread Classification Over Time chart shows workload shifts
- ▶ Permanently blocked threads flagged in Tips

[illegible]

Thread count per class across all snapshots

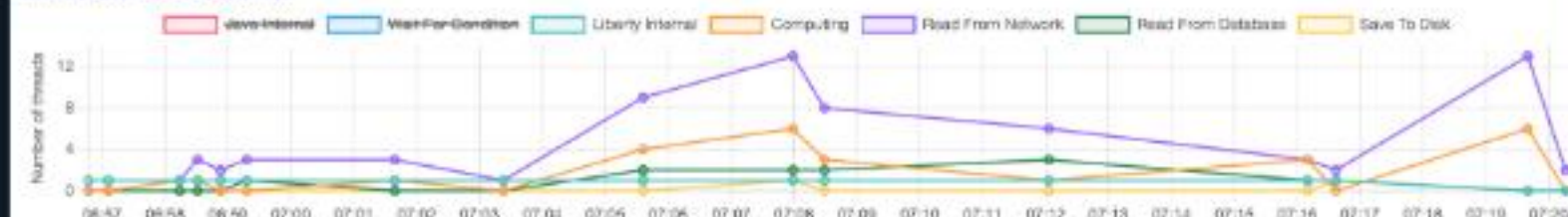
A dedicated chart shows how many threads fall into each category at every javacore timestamp — making workload shifts immediately visible.

10 classification categories

- ▶ Java Internal
- ▶ Wait For Condition
- ▶ Liberty Internal
- ▶ Display Graphics
- ▶ Read From Network
- ▶ Computing
- ▶ Read From Database
- ▶ Save To Disk
- ▶ Read From Disk
- ▶ Wait For Connection

Thread Classification Over Time chart

Shows the most used resources (database, disk, CPU) over time to detect bottlenecks.



At **7:08** and **7:20** a large number of threads are classified as **Read From Network**. This spike indicates threads waiting on a network response — an external service is a likely cause of the performance degradation observed at those times.

Tips Enhancements

WHAT

Two entry types: Warnings and Tips

- ▶ **Warning** — flags a data quality or reliability issue (e.g. corrupted javacore, too few snapshots); indicates the investigation results may not be reliable
- ▶ **Tip** — identifies a potential cause of a performance problem (e.g. high CPU thread, deadlock, long GC pause); includes Evidence, Action and Expected Impact

HOW TO USE

Open the Intelligent Tips section

- ▶ Tips are generated automatically — no extra input needed
- ▶ Click a thread name in a tip to jump to its drill-down
- ▶ Click a javacore filename to open the javacore detail page

OUTPUT

New tips added in this release

- ▶ Long GC pause tip — fires when STW pauses exceed thresholds
- ▶ Permanently blocked thread tip — instant deadlock signal
- ▶ System.exit in main thread detection

Javacore Analyser Report

from data between 2022-06-06 11:44:58 and 2022-06-06 11:59:47

[Input Files](#)

[System Information](#)

[Intelligent tips](#)

- [WARNING] The time interval between javacore [javacore.20220606.114931.32888.0009.txt](#) and [javacore.20220606.114947.32888.0010.txt](#) is 616 seconds, while the recommended maximum interval between two javacores is 300 seconds (5 minutes). It is likely that these two javacores do not correspond to one occurrence of a single issue. Please review the list of javacores and the tool only against the ones that are applicable for the issue you are investigating.
- [WARNING] The file javacore.20220606.114949.32888.0012.txt has very small size of 0 bytes. It is probably corrupted. It has been excluded from processing.
- [WARNING] The file javacore.20220606.114948.32888.0011.txt has very small size of 0 bytes. It is probably corrupted. It has been excluded from processing.
- [WARNING] 2 threads have invalid accumulated CPU. Probably one or more javacore files are corrupted.
- [WARNING] The Javacore [javacore.20220606.114947.32888.0010.txt](#) is generated on OutOfMemoryError. The signal that triggered the javacore: <<>>. You may need another tool, like Memory Analyzer Tool, to troubleshoot the issue
- [TIP] a thread [JTS Status check](#) is blocking on average 1.4 another threads per javacore. This lock might cause performance degradation.
- [TIP] The following thread is using 51% CPU: [JIT Compilation Thread-000](#). This thread might cause performance issues. Consider checking what this thread is doing
- [TIP] The verbose GC threads are using high CPU. You are likely having memory issues. Consider reviewing verbose GC for further investigation.
- [WARNING] Thread 'main' in javacore [javacore.20220606.114947.32888.0010.txt](#) contains System.exit call. This indicates the application is shutting down.
- [WARNING] Thread [JTS Status check](#) is in blocked state (B) in all 5 javacores it appears in. This thread never made progress and may be deadlocked or permanently starved. Check what lock it is waiting for.

[System resources utilization](#)

[Top 10 Blockers](#)

[All Threads](#)

[All Code](#)



Tool Integration via Plugin Architecture

WHAT

External tools can contribute report sections

A plugin system allows external diagnostic tools to inject dedicated sections into the report automatically — no manual steps, no separate viewers.

AVAILABLE NOW

Liberty message occurrence plugin

- ▶ Detects and parses Liberty server logs included in the input
- ▶ Injects a dedicated section with message severity breakdown and top message IDs
- ▶ No additional configuration required

PLANNED

Autogather integration

- ▶ Include **Autogather** output in your zip or folder
- ▶ Autogather data will appear as a dedicated section in the report

[Top 10 Blockers](#)
[All Threads](#)
[All Code](#)
[HTTP calls](#)
[Liberty System Out](#)
[What does this section tell me?](#)

Summary

MsgID	Count
Total Log Files	1
Total Messages	2

Severity Distribution

Severity	Count
Alert	0
Warning	2
Error	0
Info	0
Other	0

Top Message IDs

Message ID	Count
OVENVN7M	2

Log Files

File Name	Total Messages	Alert	Warning	Error	Info	Other
liberty.log	2	0	2	0	0	0

WatsonX AI Integration for Intelligent Tips

WHAT

LLM-generated tips and Q&A over diagnostics

A domain-aware language model will generate targeted tips and answer natural language questions over the diagnostics data. **Not yet available on Ecurep.**

HOW TO USE (PLANNED)

Ask questions in natural language

- ▶ Type questions like "show me all deadlock hints" in the report
- ▶ AI tips appear alongside rule-based tips, ranked by severity
- ▶ Thread and javacore names in AI responses auto-link to drill-downs

OUTPUT (PLANNED)

AI-enhanced Tips section

- ▶ AI-generated tips ranked HIGH / MEDIUM / LOW with Evidence & Action
- ▶ Interactive Q&A panel over the diagnostics data

[HIGH] Increase Heap Size
Evidence: Memory usage shows high memory utilization (Avg free after GC: 288126882 bytes) with max heap set to only 1G. This suggests the JVM is often unable to reclaim enough memory before triggering a garbage collection.
Action: Consider increasing the maximum heap size from 1G to at least 2-4GB, and ensure the initial heap size matches (e.g., -Xmx=3G -Xms=3G). This should reduce frequent full GCs by freeing more memory after each minor collection.
Expected Impact: Less frequent garbage collections, lower overall CPU usage during GC pauses, and improved application responsiveness.

[HIGH] Switch to G1GC with Faster Pause Goals
Evidence: Total GC pause times average 10.94ms but peak at 268.17ms (exceeding the recommended maximum of 2000ms). The GC policy is set to generational (-XXgcpolicy=gencon), which might not be optimized for this workload.
Action: Change the garbage collection policy from -XXgcpolicy=gencon to -XXgcpolicy=g1 and configure -XXMaxGCPauseMillis=500 or a lower value if feasible. This targets faster GC pause times and better throughput.
Expected Impact: Significant reduction in maximum GC pause durations, improved application latency, and more consistent performance under load.

[MEDIUM] Review JIT Compilation Thread Usage
Evidence: Top threads show significant CPU usage by JIT Compilation threads (10.78%, 8.96%) which may indicate frequent or long-running compilation cycles.
Action: Analyze the reasons for high JIT usage—e.g., method inlining, loop unrolling, and excessive use of native libraries—and consider tuning with -XXjit:4 or -XXjit:0 options to disable problematic optimizations or adjust thresholds. Also, ensure that classpath includes only necessary resources to reduce compile load.
Expected Impact: Reduced CPU contention on JIT threads, smoother application runtime, and potentially lower overall CPU usage.

[LOW] Monitor Top Threads for Blocking Patterns
Evidence: No blocking thread data is available; however, the system has experienced high CPU spikes from specific threads (e.g., [JIT Compilation Thread-000](#)).
Action: Implement detailed thread dumps or profiling to identify if these threads are causing long pauses. Consider optimizing those methods or enabling -XX:+PrintCompilation for insights into expensive compiled paths.
Expected Impact: More accurate identification of potential blocking code paths, leading to targeted optimizations and improved application throughput.

[LOW] Examine Stack Trace Patterns in Main Thread
Evidence: The main thread stack trace is dominated by the `waitForStop()` method from the OSGi framework, indicating potential long delays during shutdown or configuration changes.
Action: Review how often `waitForStop()` is called and whether it can be optimized. Consider reducing reliance on blocking calls during server startup/shutdown cycles.
Expected Impact: Faster application start/shutdown times and reduced thread contention during critical lifecycle events.

System resources utilization

CPU Load

[What does this chart tell me?](#)



Verbose GC File Visualisation

HAR File Support

AI-Based Thread Classification

Tips Enhancements

Tool Integration via Plugin Architecture

WatsonX AI Integration for Intelligent Tips

Ask natural language questions over diagnostics data → AI-generated tips and interactive Q&A in the report.

Questions

 **Join the conversation on Slack**

